

Ascend C 大模型算子优化

目 录

1 引言：大模型训练的算力瓶颈	3
2 自注意力算子的运算与复杂度	3
2.1 运算回顾	3
2.2 平方级瓶颈	3
3 GPU 上的优化算法：稀疏注意力与 FlashAttention	4
3.1 稀疏注意力	4
3.2 FlashAttention	4
3.3 FlashAttention-2	4
4 从 GPU 迁移到 NPU：Vector-Bound 问题	5
4.1 迁移的动机与挑战	5
4.2 昇腾 NPU 的计算单元	5
4.3 Vector-Bound 问题	5
4.4 NPU 上的优化方向	5
5 矩阵切分策略	6
5.1 切分流程	6
5.2 bn 切分	6
5.3 bns 切分	6
6 FlashAttention-2 前向计算分阶段详解	6
6.1 Stage 6: Dropout	7
6.2 Stage 7: 对值加权求和	7
6.3 Stage 8: Rescale	7
6.4 Softmax 接口选择	7
6.5 第二次矩阵乘法及输出合并	7
7 算子 API、编译与测试	8
7.1 两段式接口	8
7.2 获取算子 API 的两种方式	8
7.3 算子调用代码结构	8
7.3.1 固定写法部分	8
7.3.2 自适应写法部分	9
7.4 资源释放	9
7.5 编译脚本	9
7.6 编译与运行	10
8 本章你将学会	10

9 要点速查	10
10 小结	11

1 引言：大模型训练的算力瓶颈

当前所有大语言模型（如 GPT 系列）都以 **Transformer**（变换器）架构为基础。Transformer 及其衍生变体奠定了大规模模型研究的理论基础，而 GPT 正是基于 Transformer 解码器部分改进而来的变体，专注于自然语言生成。

然而，大模型的训练面临两大瓶颈：

1. **计算资源瓶颈**：模型参数和数据量急剧膨胀，对算力的需求迅速增长，同时存在计算资源利用率低的问题。
2. **内存容量瓶颈**：现有设备的内存容量往往难以满足大模型训练所需的主存规格要求。

为了提高大模型训练的效率和质量，扩展 Transformer 处理更长序列的能力、加速其计算过程成为一个亟待解决的课题。其中，注意力模块在增加序列长度方面存在明显瓶颈：它的运算时间和内存消耗会随序列长度的增加而呈平方级增长。因此，研究如何加速注意力机制的计算，对于突破大模型训练中的计算和内存瓶颈具有至关重要的意义。

直觉 不妨把自注意力想象成一场 n 人会议：每个人要听完其余所有人的发言并决定关注谁。人数翻倍，交互次数变四倍，这就是 $O(n^2)$ 的直观含义。当序列长度从 512 增到 8192，注意力矩阵从约 26 万个元素膨胀到约 6700 万个，无论是计算还是存储都承受巨大压力。

本章将带你从理解瓶颈出发，依次学习 GPU 侧的经典优化算法 FlashAttention，NPU 上的适配策略，矩阵切分方法，前向计算的分阶段实现，以及算子 API 的调用与测试全流程。

2 自注意力算子的运算与复杂度

2.1 运算回顾

自注意力的核心是三个矩阵：**查询** (Query, Q)、**键** (Key, K)、**值** (Value, V)。计算分三步：

1. 计算注意力分数： $S = QK^T$ ，得到 $n \times n$ 的分数矩阵
2. 归一化为概率： $P = \text{softmax}(S)$ ，每行元素之和为 1
3. 加权求和输出： $O = PV$ ，得到最终结果

其中 Q, K, V 均为 $n \times d$ 的矩阵， n 为序列长度， d 为头维度。

2.2 平方级瓶颈

$S = QK^T$ 产生 $n \times n$ 的矩阵，因此时间和空间复杂度均为 $O(n^2d)$ 。序列长度翻倍，计算量和内存涨 4 倍：

序列长度 n	注意力矩阵元素数	相对 $n = 512$ 增长
512	262 144	1×
1024	1 048 576	4×
2048	4 194 304	16×
8192	67 108 864	256×

这还只是一个注意力头的开销。大模型通常有几十个头，再加上多层 *Transformer* 堆叠，内存需求极为庞大。

3 GPU 上的优化算法：稀疏注意力与 FlashAttention

3.1 稀疏注意力

最直接的思路是减少注意力矩阵中需要计算的元素数，即只计算“重要”的位置。稀疏注意力分两类：

1. **基于位置的稀疏注意力**：根据位置信息确定哪些位置间的注意力权重可以忽略。例如 **Band Attention**（带状注意力），查询只关注自身上下文一定位置范围内的键，形成对角带状的注意力模式。
2. **基于内容的稀疏注意力**：根据输入内容的特征动态确定稀疏连接。例如 **Routing Transformer**（路由变换器）引入路由机制，用 k-means 算法将查询和键聚类，每个查询只关注与自身同类的键，大幅减轻计算负担。

稀疏注意力是近似算法，牺牲了全局信息交换。在需要精确结果的场景中，需要另一种思路。

3.2 FlashAttention

FlashAttention 是一种优化 I/O 访存开销的**精确**注意力算法。它不改变注意力的数学结果，而是通过减少 **HBM**（High Bandwidth Memory，高带宽内存）的读写次数来加速。三大核心手段：

1. **Tiling**（分块）：不一次性把整个 $n \times n$ 矩阵加载到显存，而是分块处理，利用速度更快但容量更小的 **SRAM**（静态随机存取存储器）替代慢速 HBM，减少访存开销。
2. **Recomputation**（重计算）：不将中间结果写回 HBM，需要使用时再次计算，以算换存，减少内存读写量。
3. **Kernel Fusion**（算子融合）：基于 Tiling，用一个 Kernel（核函数）完成 QK^T 、Softmax、 PV 整个计算流程，避免中间结果在 HBM 与 SRAM 间反复搬运。

直觉 传统做法像在厨房和仓库之间反复跑：每做一步就去仓库取食材。FlashAttention 则把食材一次搬到操作台上（Tiling 到 SRAM），一口气做完不回头（Kernel Fusion），宁可重新切菜也不回仓库取（Recomputation）。

3.3 FlashAttention-2

FlashAttention-2 在 v1 基础上进一步优化：

1. **减少非矩阵乘 FLOPs**：消除 Softmax 中冗余的 rescale（重缩放）操作，将更多计算留给高效的矩阵乘单元。
2. **序列长度方向并行**：在输入序列很长但 batch size 很小时，沿序列长度方向并行化，提升 GPU 利用率。
3. **调整内外循环遍历顺序**：进一步减少 I/O 访问开销。

例 假设序列长度 $n = 8192$, $\text{batch size} = 1$ 。FA-1 只在 batch 和头维度并行，GPU 上大量并行核心闲置。FA-2 在序列长度方向也做切分并行，让更多核心同时工作，利用率显著提升。

4 从 GPU 迁移到 NPU: Vector-Bound 问题

4.1 迁移的动机与挑战

鉴于 GPU 侧 FlashAttention-2 的成功，一种自然的思路是将其算法迁移到 NPU 侧。然而 GPU 和 NPU 硬件架构不同，迁移后针对 GPU 的并行策略可能失灵或性能降低，需要重新考虑如何适应昇腾 AI 处理器的计算单元结构。

4.2 昇腾 NPU 的计算单元

昇腾 AI 处理器有两种核心计算单元：

1. **Cube**（矩阵计算单元）：专做矩阵乘法，速度极快，是 NPU 的算力主力。
2. **Vector**（向量计算单元）：做逐元素运算，如 Softmax 中的指数、求和、缩放等，速度明显慢于 Cube。

4.3 Vector-Bound 问题

FlashAttention-2 的大部分计算（Softmax 的指数、归一化、rescale 等）由向量单元完成，而矩阵计算单元速度明显高于向量计算单元。迁移后的 FA-2 受限于 Vector 的处理能力，导致 Cube 利用率低，整体 MAC（Multiply-Accumulate，乘累加操作）利用率不高。这就是 **Vector-Bound**（向量受限）问题。

直觉 想象一条流水线，Cube 是高速冲压机，Vector 是慢速质检台。如果大部分时间在质检，冲压机只能闲着等。整体速度由最慢的环节决定，这就是 Vector-Bound。

4.4 NPU 上的优化方向

在 FA-2 基础上，可通过以下方案提升 NPU 性能：

1. **Tiling 基本块调整**：循环越多，循环间的头开销越大，性能可能越差。在满足 UB（Unified Buffer，统一缓冲区）最大空间限制的情况下，UB 切分的基本块越大，循环越少，性能越好。
2. **CV 流水并行**：由于矩阵计算和向量计算之间存在依赖（向量计算需要矩阵计算的结果，或矩阵计算需要向量计算的结果），两者可能各有空档。将注意力矩阵进一步切分，让 Cube 计算与 Vector 计算流水重叠，填补空档。
3. **核间负载均衡**：对输入进行合理切分，使每个 AI Core 计算的数据量均衡，避免部分核心空闲拖累整体。
4. **提升搬运效率**：数据的存储地址是否对齐、是否连续寻址都会影响搬运效率，需在设计时注意。

5 矩阵切分策略

矩阵切分的目的是计算每个 AI Core 计算的数据量，以及计算完这些数据所需的循环次数。此外，根据 AI Core 的 ID 和当前轮数可以计算输入数据的偏移量，方便读取正确的输入进行计算。

5.1 切分流程

1. **第一步**：根据输入形状和硬件大小限制，分别计算单核一次计算的 Q 和 KV 的序列长度，记为 s_{Outer} 和 s_{Inner} 。
2. **第二步**：判断按 **bn** 切分还是 **bns** 切分。其中 b 表示 batch， n 表示多头数， s 表示 Q 的序列长度。
3. **第三步**：依据判断结果执行 **bn** 或 **bns** 切分。
4. **第四步**：计算单核计算完所有输入需要的次数。用单核上 Q 和 K/V 的序列长度分别除以 s_{Outer} 和 s_{Inner} 并向上取整。

5.2 bn 切分

按 batch 和头数切分。设 AI Core 数为 C ，逻辑分两步：

1. 从 b 维切分：计算 b 与 C 的最大公约数 b_1 ，作为 b 维分块数，将 b 和 AI Core 按 b_1 切分。
2. 从 n 维切分：令 $n_1 = \frac{C}{b_1}$ ，取 $\frac{n}{n_1}$ 向下取整。若有余数，部分 AI Core 多处理一个头，称为主核，其余称为尾核。

5.3 bns 切分

在 **bn** 基础上再切 Q 的序列长度，逻辑分三步：

1. 从 b 维切分：计算 b 与 C 的最大公约数 b_1 。
2. 从 n 维切分：计算 n 与 $(\frac{C}{b_1})$ 的最大公约数 n_1 ，作为 n 维分块数。
3. 从 s 维切分：令 $s_1 = \frac{C}{n_1}$ 。序列长度必须为基本长度的倍数，取 $\frac{s_Q}{s_1}$ 得到每个核计算的基本序列数。若有余数，主核多处理一个基本序列，尾核少处理一个。

例 设 $b = 2, n = 5, C = 4$ (AI Core 数)，采用 **bn** 切分。

第一步： $b_1 = \gcd(2, 4) = 2$ ， b 分 2 块，每块分到 $\frac{C}{b_1} = \frac{4}{2} = 2$ 个核。

第二步： $n_1 = \frac{C}{b_1} = 2$ ， n 分 2 块，每块 $\frac{5}{2} = 2$ 向下取整，余 1。

所以每 2 个核中：1 个主核处理 3 个头，1 个尾核处理 2 个头。负载不均恰好由主核/尾核机制消化。

核间负载均衡是切分的关键：必须保证每个 AI Core 计算量尽量均衡，否则部分核心空闲会拖累整体性能。

6 FlashAttention-2 前向计算分阶段详解

前向计算分为多个 Stage (阶段)，我们聚焦其中关键的三个。

6.1 Stage 6: Dropout

训练时随机丢弃一些特征，有助于模型泛化，因为它不会过分依赖某些局部特征。推理时通常关闭 Dropout。

6.2 Stage 7: 对值加权求和

将注意力权重矩阵 P 与值矩阵 V 做矩阵乘，得到结果 O ：

$$O = PV \quad (6.1)$$

这是第二次矩阵乘法，由 Cube 单元完成，是算子中计算量最大的部分之一。

6.3 Stage 8: Rescale

这是 FA-2 减少 non-matmul FLOPs 的关键。传统 Softmax 需要反复 rescale，而 FA-2 在循环中不做 rescale，而是不断更新行的最大值 m 和行的和 d 。循环结束后，用 m 和 d 对最终结果做一次性 rescale，得到正确的 Softmax 归一化结果。

例 用小数值演示分块 Softmax 的 rescale 机制。取一行 $[1, 2, 3]$ ，分两块 $[1, 2]$ 和 $[3]$ 计算。

直接 Softmax: $e^1 + e^2 + e^3 = 2.718 + 7.389 + 20.086 = 30.193$ ，结果为 $[0.090, 0.245, 0.666]$ 。

分块计算:

块 1 $[1, 2]$: 局部最大值 $m_1 = 2$ ，计算 $e^{1-2} = 0.368$, $e^{2-2} = 1$ ，局部和 $d_1 = 1.368$ 。

块 2 $[3]$: 局部最大值 $m_2 = 3$ ，计算 $e^{3-3} = 1$ ，局部和 $d_2 = 1$ 。

合并: 全局最大值 $m = \max(2, 3) = 3$ 。块 1 的和需 rescale: $d_1 \times e^{m_1-m} = 1.368 \times e^{-1} = 1.368 \times 0.368 = 0.503$ 。全局和 $d = 0.503 + 1 = 1.503$ 。

最终结果: $\frac{0.368 \times 0.368}{1.503} \approx 0.090$, $\frac{1 \times 0.368}{1.503} \approx 0.245$, $\frac{1}{1.503} \approx 0.666$ ，与直接 Softmax 完全一致。

直觉 关键洞察：分块处理时每块只需记录局部最大值和局部和，合并时用新旧最大值的差做一次 rescale 即可。不需要保存整个 $n \times n$ 矩阵，大大节省内存。这就是 FA-2 “以算换存” 思想的精髓。

6.4 Softmax 接口选择

分块场景下的 Softmax 有两种调用方式：

1. 如果一次就能完成计算，直接调用 Softmax 接口。
2. 否则调用 SoftmaxFlashV2 接口，分首次计算和后续更新计算。
3. 最后将 Softmax 的归一化参数 rowmax 和 rowsum 写回 GM (Global Memory, 全局内存)。

6.5 第二次矩阵乘法及输出合并

得到正确的 attentionOut 分三步：

1. **计算新的 attentionOut:** 用 matmul 计算 P 和 V 的矩阵乘, P 是前一步 Softmax 接口的输出值, 具体步骤与第一次矩阵乘法相同。
2. **修正旧的 attentionOut:** 调用 Mul, 将旧的 attentionOut 与修正项 expMax 按位乘。修正项 expMax 通过 Sub 计算旧 rowmax 与新 rowmax 的差, 再调用 Exp 做指数运算得到。
3. **更新 attentionOut:** 调用 Add 将新的和修正后的 attentionOut 相加。

直觉 为什么需要修正? 因为每次循环引入新的 block 后, 全局最大值 m 可能变大, 之前累积的结果需要按 $e^{m_{\text{old}} - m_{\text{new}}}$ 缩放才能保持数值正确。这与 Stage 8 的 rescale 思想一脉相承。

7 算子 API、编译与测试

7.1 两段式接口

Ascend C 算子 API 的标准形式是**两段式接口**:

1. **第一段接口:** 计算所需的 workspace (工作空间) 大小。
2. **第二段接口:** 执行真正的计算。

漏掉第一段会导致 workspace 空间不足甚至崩溃, 两段缺一不可。

7.2 获取算子 API 的两种方式

1. **调用算子库现成算子:** 直接使用 aclnnFlashAttentionScore。配置环境时, 除安装 toolkit 外, 还需安装 CANN 算子二进制软件包。
2. **编译自己编写的代码:** 编写 aclnnFlashAttentionScore 的算子代码, 编译生成算子 API。

7.3 算子调用代码结构

测试项目主要包含两个文件: 算子调用代码 (*.cpp) 和编译脚本 (CMakeLists.txt)。调用代码分为固定写法和自适应写法两部分。

7.3.1 固定写法部分

可在官网或相关书籍查到, 主要包括 AscendCL 初始化函数、创建 aclTensor 函数、打印输出结果函数, 以及主函数中用于 AscendCL 初始化的变量创建和释放。基本框架如下:

```
aclInit(nullptr);
aclrtContext context;
aclrtCreateContext(&context, deviceId);

aclrtStream stream;
aclrtCreateStream(&stream);

aclTensor* query = aclCreateTensor(shapeQ, dtype, format, stridesQ);
aclTensor* key   = aclCreateTensor(shapeK, dtype, format, stridesK);
aclTensor* value = aclCreateTensor(shapeV, dtype, format, stridesV);
aclTensor* output = aclCreateTensor(shapeO, dtype, format, stridesO);
```

逐行说明：aclInit 初始化 AscendCL 运行环境；aclrtCreateContext 创建设备上下文，将后续操作绑定到指定设备；aclrtCreateStream 创建异步计算流；aclCreateTensor 根据输入的形状、数据类型、格式和步长创建 aclTensor 句柄，供后续算子使用。

7.3.2 自适应写法部分

根据具体算子需求编写，主要分为四步：

1. 构造输入输出（上一节的 aclCreateTensor）
2. 调用算子 API 的**第一段接口**计算所需 workspace 大小
3. 调用算子 API 的**第二段接口**执行计算
4. 释放输入输出

```
uint64_t workspaceSize = 0;
aclOpExecutor* executor = nullptr;

aclnnFlashAttentionScoreGetWorkspaceSize(
    query, key, value, output, &workspaceSize, &executor);

void* workspace = nullptr;
aclrtMalloc(&workspace, workspaceSize, ACL_MEM_MALLOC_HUGE_FIRST);

aclnnFlashAttentionScore(workspace, workspaceSize, executor, stream);
aclrtSynchronizeStream(stream);
```

逐行说明：aclnnFlashAttentionScoreGetWorkspaceSize 是第一段接口，传入输入输出 Tensor，返回所需的 workspace 大小和 executor 句柄；aclrtMalloc 根据 workspaceSize 在 device 侧分配工作空间；aclnnFlashAttentionScore 是第二段接口，传入 workspace 和 executor 执行实际计算；aclrtSynchronizeStream 阻塞等待流中所有任务完成，确保结果已写回。

7.4 资源释放

计算完成后，必须释放 aclTensor 变量和 device 侧内存，顺序不能错：

```
aclDestroyTensor(query);
aclDestroyTensor(key);
aclDestroyTensor(value);
aclDestroyTensor(output);

aclrtFree(workspace);
aclrtDestroyStream(stream);
aclrtDestroyContext(context);
aclFinalize();
```

逐行说明：aclDestroyTensor 销毁通过 aclCreateTensor 创建的变量；aclrtFree 释放通过 aclrtMalloc 申请的 device 侧内存；aclrtDestroyStream 和 aclrtDestroyContext 分别销毁流和上下文；aclFinalize 去初始化 AscendCL。先销毁 Tensor 再释放内存，顺序不能反，否则可能内存泄漏。

7.5 编译脚本

编写编译脚本（CMakeLists.txt）时：

1. 如果调用算子库中的现成 API，直接复制官网或相关书籍中编译脚本的代码即可。
2. 如果通过编译自己编写的代码获得 API，需在复制的基础上做两处调整：
 - 设置可执行文件名，额外设置包含算子 API 的 cpp 文件，AUTOGEN_PATH 为该 cpp 文件的路径
 - 添加头文件搜索路径，额外添加 AUTOGEN_PATH

```
set(EXECUTABLE_NAME attention_score_test)
set(AUTOGEN_PATH ${CMAKE_CURRENT_SOURCE_DIR}/op_kernel)
include_directories(${AUTOGEN_PATH})
```

第一行设置可执行文件名；第二行将 AUTOGEN_PATH 指向包含算子 API 定义的 cpp 文件目录；第三行将该目录加入头文件搜索路径，使编译器能找到自动生成的 API 头文件。

7.6 编译与运行

提前准备好测试项目（算子调用代码和编译脚本），执行编译命令完成构建：

```
mkdir build && cd build
cmake .. && make
```

运行成功后，终端会依次打印输出中的每个值。

8 本章你将学会

1. 理解自注意力算子 $O(n^2)$ 复杂度瓶颈对大模型训练的影响
2. 区分稀疏注意力（近似）与 FlashAttention（精确）两类优化路线
3. 解释 FlashAttention 的三大手段：Tiling、Recomputation、Kernel Fusion
4. 分析 FA-2 迁移到 NPU 后的 Vector-Bound 问题及四种 NPU 优化方向
5. 掌握 bn 与 bns 两种矩阵切分策略，以及主核/尾核负载均衡机制
6. 理解 FA-2 前向计算中 Stage 8 的延迟 rescale 原理与分块 Softmax 合并机制
7. 编写并测试 acLnnFlashAttentionScore 两段式 API 调用代码

9 要点速查

要点	说明
注意力复杂度	$O(n^2d)$ ，序列翻倍计算量涨 4 倍
Band Attention	基于位置，查询只关注上下文带状范围内的键
Routing Transformer	基于内容，k-means 聚类查询和键，同类内交互
FlashAttention 三招	Tiling（SRAM 替 HBM）+ Recomputation（以算换存）+ Kernel Fusion（单 Kernel）
FA-2 优化	减少 non-matmul FLOPs / 序列方向并行 / 调循环顺序
Vector-Bound	FA-2 在 NPU 上向量单元受限，Cube 利用率低，MAC 不高
CV Pipeline	切分注意力矩阵让 Cube 与 Vector 计算流水重叠
Tiling 块调整	UB 内基本块越大循环越少，性能越好
核间负载均衡	合理切分使每核数据量均衡
bn 切分	按 batch 加头数切分，gcd 求 b_1 ，主核/尾核处理余数
bns 切分	再切 Q 序列长度，必须为基本长度倍数
Stage 7	$O = PV$ ，注意力权重与值矩阵乘

Stage 8 Rescale	循环内不做 rescale，循环后用 m 和 d 统一 rescale
SoftmaxFlashV2	分块 Softmax：首次计算加后续更新，最后写回 rowmax 和 rowsum
两段式 API	第一段算 workspace 大小，第二段执行计算
资源释放	先 <code>aclDestroyTensor</code> 再 <code>aclrtFree</code> ，顺序不能反

10 小结

大模型算子优化的核心是突破自注意力的 $O(n^2)$ 瓶颈。FlashAttention 通过 Tiling、Recomputation 和 Kernel Fusion 三大手段在 GPU 上取得了巨大成功，但直接迁移到 NPU 会遇到 Vector-Bound 问题：FA-2 大量计算由较慢的向量单元完成，高速的矩阵单元反而闲置。解决之道是针对昇腾 Cube 与 Vector 分离架构重新设计算法，包括 CV 流水并行、Tiling 基本块调整、核间负载均衡和搬运效率优化。

在实现层面，矩阵切分（bn 与 bns）决定了数据如何分配到各 AI Core，前向计算的 Stage 6 至 Stage 8 展示了 Dropout、加权求和与延迟 rescale 的完整流程。最后，通过两段式 API 调用模式，我们可以将自注意力算子集成到实际的 Ascend C 项目中并完成测试。这是从“调库使用者”迈向“高性能算子开发者”的必经之路。

讲义基于华为 CANN「第六章 Ascend C 大模型算子优化」课程内容编写